

# TypeScript Interview Questions — Simple Answers (Say This in Interview)

Plain English. Just Question + Answer + a tiny example. No heavy textbook words.

## Table of Contents

**Basics:** [1](#) · [2](#) · [3](#) · [4](#)

**Types:** [5](#) · [6](#) · [7](#) · [8](#) · [9](#) · [10](#) · [11](#) · [12](#) · [13](#)

**Objects & Classes:** [14](#) · [15](#) · [16](#) · [17](#) · [18](#) · [19](#) · [20](#) · [21](#)

**Functions:** [22](#) · [23](#)

**Generics & Advanced:** [24](#) · [25](#) · [26](#) · [27](#) · [28](#) · [29](#) · [30](#) · [31](#) · [32](#)

---

## 1. What is TypeScript?

**Answer:** TypeScript is just JavaScript with types added on top. It's not a new language — your TypeScript code gets converted into normal JavaScript before it runs. The main benefit: it catches

mistakes while you're typing the code, instead of finding out only when the app crashes.

```
function add(a: number, b: number): number
{
    return a + b;
}
add("5", 10); // Error caught immediately,
before running
```

---

## 2. TypeScript vs JavaScript

**Answer:** JavaScript doesn't check types — you only find out about a mistake when the app actually runs and breaks. TypeScript checks types while you write the code, gives better autocomplete in your editor, and makes it easier to work in a big codebase because you always know what shape of data a function expects.

---

## 3. Type Inference

**Answer:** This means TypeScript can automatically figure out the type of a variable on its own, just by looking at the value you gave it — you don't always have to write the type manually.

```
let name = "Asha"; // TS automatically
knows this is a string
// name = 5; -> Error, even though we never
wrote ": string"
```

---

## 4. Basic Types in TypeScript

**Answer:** The common basic types are `string`, `number`, `boolean`, `array`, `object`, and `null / undefined`. You can write the type after a colon `:`.

```
let username: string = "Asha";
let age: number = 25;
let isActive: boolean = true;
let scores: number[] = [10, 20, 30];
```

---

## 5. any vs unknown vs never vs void

**Answer:** `any` turns off type checking completely — try to avoid it. `unknown` can hold any value too, but you must check what it actually is before using it, so it's safer than `any`. `void` means a function doesn't return anything useful. `never` means a function never actually finishes normally — it either always throws an error or loops forever.

```
let val: unknown = 5;
if (typeof val === "number") {
  console.log(val + 1); // safe, we checked
  first
}

function logMsg(msg: string): void {
  console.log(msg); }
function fail(msg: string): never { throw
  new Error(msg); }
```

---

## 6. Union Types

**Answer:** A union type means a value can be **one of several types**. We use `|` (pipe) between the types.

```
type ID = string | number;
let userId: ID = 101;
userId = "U101"; // also valid
```

---

## 7. Intersection Types

**Answer:** An intersection type combines multiple types into one — the value must satisfy **all** of them at once. We use `&`.

```
type Name = { name: string };
type Age = { age: number };
```

```
type Person = Name & Age; // must have both
const p: Person = { name: "Asha", age: 25
};
```

---

## 8. Type Alias

**Answer:** A type alias is just a name we give to a type, so we can reuse it instead of writing the same shape again and again.

```
type Status = "loading" | "success" |
"error";
let currentStatus: Status = "loading";
```

---

## 9. Interface

**Answer:** An interface describes the shape of an object – what properties it should have and what type each property is.

```
interface User {
  id: number;
  name: string;
}
const user: User = { id: 1, name: "Asha" };
```

---

## 10. Interface vs Type Alias

**Answer:** Both can describe an object's shape, and most of the time they work the same way. The difference: an `interface` can be reopened later to add more fields, and it can be extended with `extends`. A `type` can't be reopened, but it can do things an interface can't, like unions ( `"a" | "b"` ). A simple rule: use `interface` for object/API models, use `type` for unions.

---

## 11. Literal Types

**Answer:** A literal type means the value can only be one **exact** value (or one from a fixed small list), not just "any string" or "any number."

```
let direction: "up" | "down" | "left" |  
"right";  
direction = "up"; // valid  
// direction = "north"; -> Error
```

---

## 12. Tuples

**Answer:** A tuple is an array with a **fixed number of items**, where each position has its own specific type — unlike a normal array where every item is usually the same type.

```
type UserTuple = [number, string]; // [id, name]
const user: UserTuple = [1, "Asha"];
```

Fun fact: `useState` in React actually returns a tuple — that's why we can destructure it as `[value, setter]`.

---

### 13. Enums

**Answer:** An enum is a way to give names to a fixed set of values, so the code is easier to read instead of using random strings/numbers everywhere.

```
enum OrderStatus {
  Pending = "PENDING",
  Shipped = "SHIPPED",
  Delivered = "DELIVERED",
}
let status: OrderStatus =
  OrderStatus.Shipped;
```

### 14. Optional Properties ( ? )

**Answer:** Adding a `?` after a property name means that property is **not required** — the object can be created with or without it.

```
interface User {  
  name: string;  
  nickname?: string; // optional  
}  
const u: User = { name: "Asha" }; // fine  
without nickname
```

---

## 15. readonly

**Answer:** `readonly` means a property can be set once (usually when the object is created), but it can **never be changed afterward**.

```
interface User {  
  readonly id: number;  
}  
const u: User = { id: 1 };  
// u.id = 2; -> Error
```

---

## 16. Index Signatures

**Answer:** An index signature lets you describe an object where you don't know the exact property names in advance, but you know what type the keys and values will be.

```
interface Scores {  
  [studentName: string]: number;
```



```
}  
const marks: Scores = { Asha: 90, Ravi: 85  
};
```

---

## 17. Classes in TypeScript

**Answer:** TypeScript classes work like JavaScript classes, but you can also add types to properties and method parameters.

```
class Person {  
  name: string;  
  constructor(name: string) {  
    this.name = name;  
  }  
  greet(): string {  
    return `Hi, I'm ${this.name}`;  
  }  
}
```

---

## 18. Access Modifiers (public, private, protected)

**Answer:** These control who can use a class property/method from outside. `public` (default) means anyone can access it. `private` means only that class itself can use it. `protected` means that

class and any class that extends it can use it, but nothing outside.

```
class Account {  
    private balance: number = 0;  
    protected accountType: string =  
    "savings";  
    public ownerName: string = "Asha";  
}
```

---

## 19. extends vs implements

**Answer:** `extends` is for inheritance — a class or interface builds on top of another one and reuses its features. `implements` is used on a class to force it to follow an interface's structure — meaning it must include everything that interface promises.

```
interface Shape {  
    area(): number;  
}  
class Circle implements Shape {  
    constructor(private radius: number) {}  
    area() { return Math.PI * this.radius **  
    2; }  
}
```

---

## 20. Abstract Classes

**Answer:** An abstract class is a class you can't create directly with `new` — it's meant to be extended by other classes. It can have "abstract methods," which have no actual code in them; any class extending it **must** provide that method's logic.

```
abstract class Animal {  
    abstract makeSound(): void; // no body,  
    child must implement  
}  
class Dog extends Animal {  
    makeSound() { console.log("Woof"); }  
}
```

---

## 21. Static Members

**Answer:** A `static` property or method belongs to the **class itself**, not to individual objects created from it. You access it directly using the class name, without creating an instance.

```
class MathHelper {  
    static PI = 3.14;  
    static square(n: number) { return n * n;  
}  
}  
console.log(MathHelper.PI); // no "new"  
needed
```

---

## 22. Function Types & Optional/Default Parameters

**Answer:** We can describe what a function's input and output types should look like. A parameter with `?` is optional. A parameter can also have a default value, which is used if nothing is passed.

```
function greet(name: string, greeting:
string = "Hello"): string {
    return `${greeting}, ${name}`;
}

function logSomething(message?: string) {
    console.log(message ?? "no message");
}
```

---

## 23. Function Overloading

**Answer:** This means writing multiple "versions" of a function signature for the same function name, so TypeScript knows exactly what return type to expect depending on what kind of arguments you pass in.

```
function getValue(id: number): string;
function getValue(id: string): number;
function getValue(id: any): any {
    return typeof id === "number" ? "result-
```

```
string" : 123;  
}
```

---

## 24. Generics

**Answer:** Generics let you write a function/interface/component that can work with **any type**, while still staying type-safe — instead of writing the same logic again for every type, or using `any` and losing safety. `T` is just a placeholder name for "whatever type gets used."

```
function getFirst<T>(arr: T[]): T {  
    return arr[0];  
}  
getFirst<number>([1, 2, 3]); // returns a  
number  
getFirst<string>(["a", "b"]); // returns a  
string
```

---

## 25. Generic Constraints

**Answer:** Sometimes we want a generic to only accept types that have certain properties — we do this using `extends` inside the generic.

```
function printLength<T extends { length:  
number }>(item: T): void {
```

```
    console.log(item.length);
  }
  printLength("hello"); // works, strings
  have .length
  printLength([1, 2, 3]); // works, arrays
  have .length
  // printLength(5); -> Error, numbers don't
  have .length
```

---

## 26. keyof

**Answer:** `keyof` takes an object type and gives you back a union of just its **property names** as a type.

```
interface User { id: number; name: string;
}
type UserKeys = keyof User; // "id" |
"name"

function getValue(user: User, key:
UserKeys) {
  return user[key];
}
```

---

## 27. typeof (in Type Context)

**Answer:** This is different from JavaScript's `typeof`. Here, it lets you take an **already existing variable** and

reuse its shape as a TypeScript type, so you don't have to write the same shape twice.

```
const defaultUser = { id: 1, name: "Asha"
};
type UserType = typeof defaultUser; // {
id: number; name: string }
```

---

## 28. Utility Types: Partial, Required, Pick, Omit, Record

**Answer:** These are built-in helpers that change an existing type without rewriting it. `Partial<T>` makes every field optional. `Required<T>` makes every field mandatory. `Pick<T, K>` keeps only the fields you choose. `Omit<T, K>` removes the fields you choose. `Record<K, V>` builds an object type where every key is type `K` and every value is type `V`.

```
interface User { id: number; name: string;
email: string; }

type UserUpdate = Partial<User>;
// good for an edit form
type UserPreview = Pick<User, "id" |
"name">; // only id and name
type NoEmail = Omit<User, "email">;
// everything except email
```

---

## 29. Type Assertion ( `as` )

**Answer:** This is when you tell TypeScript "trust me, I know this value's real type," overriding what TypeScript thinks it is. It doesn't actually check anything at runtime — if you're wrong, it can cause bugs. Use it carefully, only when you're sure.

```
const data = response.data as User;
```

---

## 30. Type Guards & Type Narrowing

**Answer:** A type guard is a real runtime check (like `typeof` or `instanceof` ) that confirms what type a value actually is. "Narrowing" means TypeScript automatically becomes smarter about a variable's type inside an `if` block once you've checked it. This is the **safe** version of `as` , because it's actually verified, not just assumed.

```
function printValue(value: string | number)
{
  if (typeof value === "string") {
    console.log(value.toUpperCase()); // TS
    now knows it's a string here
  } else {
    console.log(value.toFixed(2)); // TS
    now knows it's a number here
  }
}
```



```
}  
}
```

---

### 31. Non-null Assertion ( ! )

**Answer:** Adding `!` after a value tells TypeScript "I'm sure this is not `null` or `undefined`, stop warning me about it." It doesn't actually remove `null / undefined` at runtime — it just stops the TypeScript error. If you're wrong, the app can still crash, so use it carefully.

```
function getUser(id: number): User |  
undefined { /* ... */ }  
const user = getUser(1)!; // "I promise  
this exists"  
console.log(user.name);
```

---

### 32. Modules & Declaration Files (.d.ts)

**Answer:** Every TypeScript file that uses `import / export` is treated as its own module, with its own private scope. A `.d.ts` file (declaration file) contains only type information, no real code — it's used to describe types for JavaScript libraries that weren't originally written in TypeScript, so we still get autocomplete and type checking when using them.

```
// utils/formatPrice.ts
export function formatPrice(price: number):
string {
    return `₹${price.toFixed(2)}`;
}

// another file
import { formatPrice } from
"./utils/formatPrice";
```

---

[← Back to top](#)